รายงานสูตรการคำนวณ Productivity

วิเคราะห์โค้ดใน `smart-backend-88-62/services/productivity/` โดยเน้นสูตร Register, Actual, Leave, OT, Man-hour, Productivity และการรวมผลรายช่วงเวลา

วันที่จัดทำ: 8 ตุลาคม 2026 **Branch:** Yo (commit e7e64f5) **ขอบเขต:** 11 ไฟล์, ราว 7,300 บรรทัด

วิธีการ: อ่านโค้ด (ไม่ได้รันกับฐานข้อมูลจริง)

---

8 / 3 / 11
ชั่วโมงต่อคน: งานปกติ / OT1 วันทำงาน / OT2 วันหยุด (ใช้ใน Period Summary)

pcs ÷ MH
Productivity = ยอด Output (ชิ้น) ÷ Man-hour รวมของช่วงเวลา

16
ข้อสังเกตที่พบ: 4 ระดับสูง, 7 ระดับกลาง, 5 ระดับต่ำ (ดูหัวข้อที่ 9)

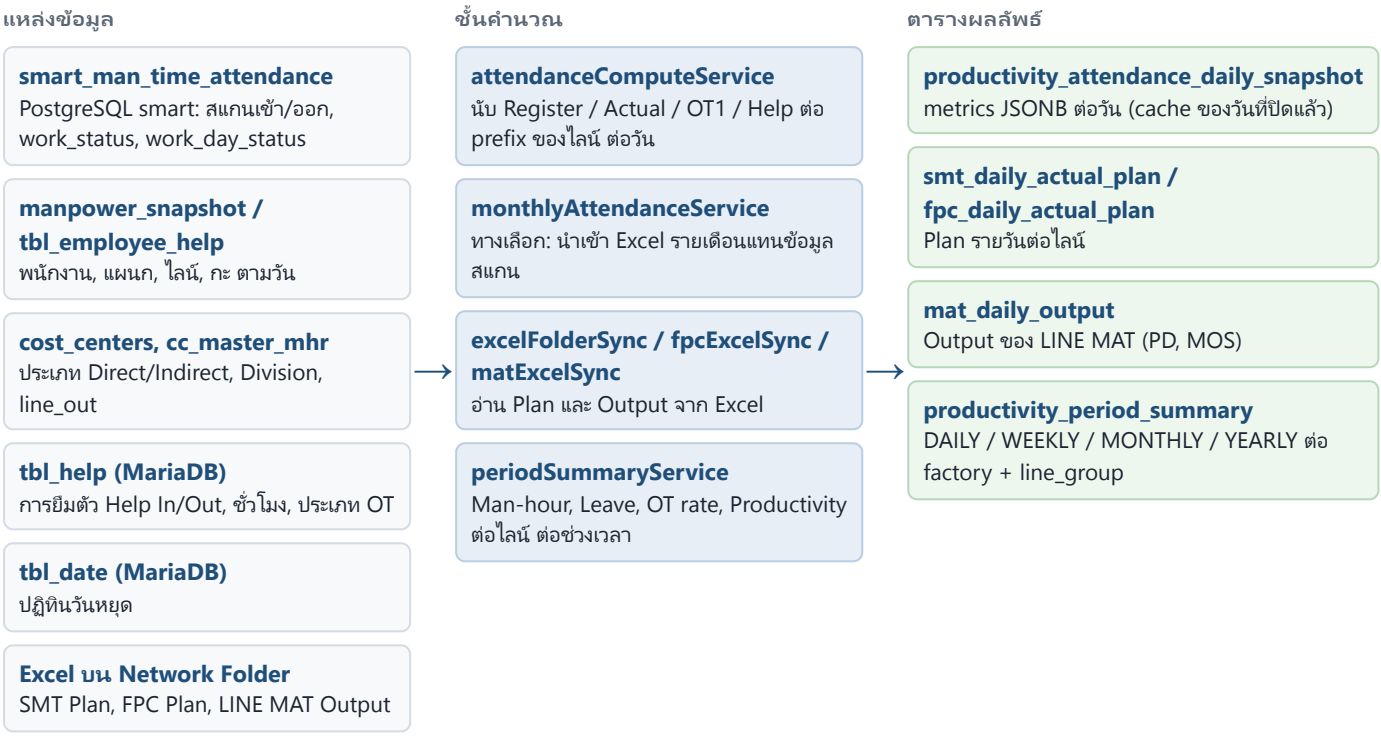
สรุปสำหรับผู้บริหาร

- สูตรหลักของ Man-hour รายไลน์รายวันคือ `(Actual + HelpIn − HelpOut) × 8 + (OT1 + HelpInOT1 − HelpOutOT1) × 3` ในวันทำงาน และ `(OT2 คน + HelpInOT2 − HelpOutOT2) × 11` ในวันหยุด สูตรนี้อยู่ใน `periodSummaryService.js` และตรงกับสูตรฝั่ง Frontend (`useMatrixAggregation.ts`) ในระดับไลน์ย่อย
- Productivity คิดเป็น **ชิ้นต่อ Man-hour** ทั้งระดับช่วงเวลา และแบบสะสม: รายวันสะสมตั้งแต่ต้นเดือน (MTD) ส่วนรายสัปดาห์และรายเดือนสะสมตั้งแต่ต้นปีงบประมาณ 1 เมษายน (FYTD)
- **ปัญหาที่ควรแก้ก่อน:** ไฟล์ `outputSnapshotService.js` เป็นสำเนาของ `monthlyAttendanceService.js` ทุกไบต์ จึงไม่มีฟังก์ชัน `syncDailyOutputSnapshotFromPostgres` ที่ Route และ Scheduler เรียกใช้ การซิงก์ Output Snapshot จึงทำงานไม่ได้ และ API บางตัวตอบ 500 ทั้งที่บันทึกข้อมูลสำเร็จแล้ว
- ค่าคงที่และนิยามไม่ตรงกันระหว่างจุดคำนวณ: OT1 คิด 2.5 ชม. ใน Snapshot แต่คิด 3 ชม. ใน Period Summary, วิธีนับ Help ในการนำเข้ารายเดือนเป็น "ชั่วโมง" แต่ในการคำนวณสดเป็น "จำนวนคน", Register ของ Direct FPC นับผู้จัดการแต่ Actual ไม่นับ

สารบัญ

1. ภาพรวมการไหลของข้อมูล
2. หน้าที่ของแต่ละไฟล์
3. สูตรระดับพนักงาน (Attendance Compute)
4. สูตรระดับไลน์ต่อวัน (Man-hour, Leave, OT)
5. การรวมกลุ่ม Macro / Direct
6. Output และ Plan
7. การรวมผลรายช่วงเวลา (Period Summary)
8. Parser ของ Excel และ Snapshot
9. ข้อสังเกตและความเสี่ยง
10. ภาคผนวก: ค่าคงที่และกฎการปัดเศษ

1. ภาพรวมการไหลของข้อมูล



หมายเหตุ: `output_daily_snapshot` (Output จาก MES) ถูกอ่านใน Period Summary แต่ฟังก์ชันที่ควรเติมข้อมูลตารางนี้หายไป (ดูข้อสังเกต H1)

ลำดับการคำนวณ

1. **ระดับพนักงาน:** แต่ละแถวการสแกนถูกจัดประเภทเป็น Register, Actual และ OT1 แล้วนับเข้า "prefix" ของไลน์ที่พนักงานสังกัด (เช่น `line_c`, `mota`, `asy2`) และนับเข้ายอดรวมโรงงาน (`total_*`, `fpc_*`)
2. **ระดับไลน์ต่อวัน:** แปลงจำนวนคนเป็น Leave, Man-hour และ OT ด้วยค่าคงที่ 8, 3 และ 11 ชั่วโมง
3. **ระดับกลุ่ม:** รวมไลน์ย่อยเป็นกลุ่มที่แสดงบน UI (LINE B, Macro SMT_F, Direct SMT ฯลฯ)
4. **ระดับช่วงเวลา:** รวมรายวันเป็นรายสัปดาห์ รายเดือน และรายปี แล้วคำนวณค่าเฉลี่ย อัตรา และ Productivity เพื่อเขียนลง `productivity_period_summary`

2. หน้าที่ของแต่ละไฟล์

ไฟล์	บรรทัด	หน้าที่ / สูตรที่เกี่ยวข้อง
`attendanceComputeService.js`	1,043	คำนวณ metrics รายวันจากข้อมูลสแกนจริง (Register, Actual, OT1, Help) และ cache ลง `productivity_attendance_daily_snapshot`
`attendanceSyncService.js`	289	ตรวจว่าวันไหนข้อมูลสแกนใหม่กว่า snapshot (เลือก batch ที่มีจำนวนแถวมากที่สุด) แล้วคำนวณซ้ำเฉพาะวันนั้น และ rollup period summary
`periodSummaryService.js`	1,342	หัวใจของสูตร: Man-hour, Leave, OT, Macro groups, Output/Plan mapping, Productivity, Week key
`monthlyAttendanceService.js`	942	นำเข้าไฟล์ Excel การลงเวลารายเดือนและเขียนทับ snapshot รายวัน (กฎ Actual และ Help ต่างจากการคำนวณสด)
`outputSnapshotService.js`	942	**เนื้อหาเหมือน `monthlyAttendanceService.js` ทุกไบต์** (MD5 `8816a8f2…` ตรงกัน) จึงไม่มีฟังก์ชันซิงก์ Output

`snapshotManager.js`	742	Snapshot พนักงานรายวัน, กฎ cutoff 09:00, การเก็บข้อมูลราว 45 วัน, อ่าน/เขียน `output_daily_snapshot`, Scheduler รุ่นเก่า
`snapshotScheduler.js`	72	Scheduler รุ่นใหม่ (09:00 ทุกวัน) แต่ไม่มีที่ใดเรียก `initSnapshotScheduler()`
`excelFolderSync.js`	470	อ่าน SMT Daily Output Monitoring (Plan รายวันต่อไลน์ SMT)
`fpcExcelSync.js`	946	อ่าน Plan EFPC (pcs / sht / lot) ด้วยการค้นหาหัว Section และให้คะแนนแถว
`fpcPlanMapping.js`	193	นิยาม 16 รายการ Plan ของ FPC (ชีท, คำค้น Section, คำยกเว้น)
`matExcelSync.js`	344	อ่าน "File Scan mat" ชีท Output Report → PD และ MOS ของ LINE MAT

3. สูตรระดับพนักงาน (attendanceComputeService)

ฟังก์ชัน `computeLiveAttendanceData(startDate, endDate)` วนทุกแถวของ `smart_man_time_attendance` ในช่วงวันที่ (หลังกรองแถวค้างด้วย majority filter) แล้วสะสมตัวนับต่อวัน

3.1 การกรองข้อมูลก่อนคำนวณ

- **Majority filter** `attendanceComputeService.js:128-165`: ในแต่ละวัน เก็บเฉพาะแถวที่ `updated_date` ตรงกับค่าที่พบบ่อยที่สุดของวันนั้น เพื่อตัดแถวค้างจาก batch เก่า
- **Loan exclusion** `:285-296, :671`: พนักงานใน `manpower_loan_exclude` ที่ active และวันที่อยู่ในช่วง `[start_date, end_date]` ถูกข้ามทั้งแถว (ไม่นับทั้ง Register และ Actual)

3.2 การจัดประเภทพนักงาน

```
deptCodeOT = department.split('/')[0].split(' ')[0].replace('-', '')
type = cost_centers.type  ?? cc_master_mhr.mhr_cc_type  ?? 'NOT_FOUND'
// บังคับเป็น Direct ถ้าเข้าเงื่อนไขใดเงื่อนไขหนึ่ง
if deptCodeOT ∈ {P4613A/B/D, P4623A/B/D, P4631A/B/D, P4633A/B/D, P4662A/B/D, P4663A/B/D, P4671D,
P5401B}
   or line ∈ {HOT PRESS/A, /B, /D}
   or deptCodeOT ขึ้นต้นด้วย P461…P468           → type = 'Direct'
div = 'E-FPC' ถ้า deptCodeOT ขึ้นต้นด้วย P460…P468, ไม่เช่นนั้นใช้ mhr_div
lineOut = cost_centers.line_out ?? (line + '/' + shift)
```

อ้างอิง `:301-370` ไลน์ที่ใช้นับคือ `lineOut` ซึ่งถูกแปลงเป็นหลาย prefix พร้อมกันผ่าน `getAllPrefixesForLine()` `:598-648` เช่น `AT_FRONT/A` นับทั้ง `line_a_auto` และ `at_front` ส่วน `MAS/A` นับทั้ง `mas` และ `mas_rew` นอกจากนี้ยังเติม prefix อัตโนมัติจากชื่อไลน์ (`LINE C/A` → `line_c`, `c`) และใช้ Set กันนับซ้ำ

3.3 Register, Actual, OT1

```
// Register: นับ 1 ครั้งต่อ (วัน, พนักงาน) ที่แถวแรกที่พบ
Register += 1  เมื่อเป็นแถวแรกของพนักงานในวันนั้น

// Actual (มาทำงานจริง)
hoursWorked = (scan_time_out − scan_time_in) / 60 นาที
if 0 < hoursWorked < 4.0                    → Actual = false   (ทำงานไม่ถึง 4 ชม.)
else if work_day_status มี 'H' หรือ 'O'     → Actual = มี scan_in หรือ scan_out
else                                          → Actual = work_status ∈ {'Normal','Abnormal'}

// OT1 (นับเฉพาะคนที่ Actual และมี scan_time_out)
กะกลางคืน ถ้า ชั่วโมง scan_in อยู่ในช่วง 18–23 (ถ้าไม่มี scan_in ให้ดูกะ N หรือ B)
กะกลางคืน: OT ถ้าออก 07:30–12:59
กะกลางวัน: OT ถ้าออก 19:30–23:59
OT1 += 1
```

อ้างอิง `:675-693` (Actual), `:754-787` (Register), `:827-876` (OT1)

3.4 ยอดย่อยที่ถูกสะสม

คีย์ใน metrics	ความหมาย / กฎ
`{prefix}_total_register` / `_total_actual` / `_ot1_actual`	จำนวนคนต่อ prefix ของไลน์
`{prefix}_total_man_hour`	+8 ต่อ Actual และ +2.5 ต่อ OT1 `:793, :853` (Period Summary ไม่ได้ใช้ค่านี้)
`cc_{dept}_*`, `{dept}_*`	ยอดเดียวกันแยกตาม Cost Center (ทั้งรหัสดิบและรหัสที่ตัด `-`, `_`)
`direct_*` / `indirect_*` / `indirectProduction_*` / `unknown_*`	แยกตามประเภทพนักงาน
`fpc_total_*`	Division E-FPC และไม่ใช่แผนก `(MGR)`
`fpc_direct_register`	E-FPC และ Direct **(นับ MGR ด้วย)** `:778`
`fpc_direct_actual` / `_ot1_actual`	E-FPC, Direct และ **ไม่ใช่ MGR** `:817, :871`
`work_day_type`	ส./อา. → 'H' ถ้า OT1 ≥ 20 คน ไม่เช่นนั้น 'O'; วันธรรมดา → 'W' `:880-893`

3.5 Help In / Help Out (tbl_help)

```
// ทุกแถวที่ hour > 0 (ไม่ว่า type ใด)
help_in_normal  += hour / 8      (แปลงชั่วโมงเป็น "คนเทียบเท่า")
help_out_normal += hour / 8
help_in_psn      = จำนวน id_code ไม่ซ้ำ

// แยก OT ตาม type (ตัดช่องว่าง, ตัวพิมพ์ใหญ่)
type มี 'OT1' หรือ 'WORKING'  → help_*_ot1 = จำนวน id_code ไม่ซ้ำ
type มี 'OT2' หรือ 'HOLIDAY'  → help_*_ot2 = จำนวน id_code ไม่ซ้ำ
```

อ้างอิง `:916-1023` Help In นับเข้า prefix ของ `line_in` และ Help Out นับเข้า prefix ของ `line_out` ในระดับรวมโรงงาน Help In กับ Help Out มีค่าเท่ากันเสมอ จึงหักล้างกันใน Macro PCN

3.6 Cache และการคำนวณสด

`getAttendanceSummaryData()` `:171-229` อ่านวันที่ก่อนวันนี้จาก snapshot ก่อน ถ้าวันไหนไม่มีจะคำนวณสดเป็นช่วงต่อเนื่องจากวันแรกที่ขาดถึงวันสุดท้ายที่ขาด และวันที่ผ่านไปแล้วจะถูกบันทึกกลับลง snapshot โดยไม่รอผล (fire-and-forget) ส่วนวันนี้คำนวณสดทุก

ครั้ง

4. สูตรระดับไลน์ต่อวัน (periodSummaryService · calcSubline)

สำหรับแต่ละวัน ระบบหาก่อนว่าเป็นวันทำงานหรือไม่ แล้วคำนวณค่า 7 ตัวต่อ prefix `periodSummaryService.js:124-153`

```
// วันทำงาน
isWorking = 0 ถ้า tbl_date.w_result = 'holiday', ถ้าไม่มีในปฏิทินให้ใช้ (วันอาทิตย์ ? 0 : 1)

leave       = isWorking ? max(0, register − actual) : 0
normalHours = isWorking ? max(0, (actual + helpInNormal − helpOutNormal) × 8) : 0
ot1Hours    = isWorking ? max(0, (ot1Actual + helpInOT1 − helpOutOT1) × 3) : 0
ot2Psn      = !isWorking ? (ot1Actual > 0 ? ot1Actual : (actual < register ? actual : 0)) : 0
ot2Hours    = !isWorking ? max(0, (ot2Psn + helpInOT2 − helpOutOT2) × 11) : 0

manHours    = round(normalHours + ot1Hours + ot2Hours)
otPsn       = isWorking ? ot1Actual : ot2Psn
otWorking   = isWorking ? ot1Actual : 0
otHoliday   = !isWorking ? ot2Psn : 0
```

ตัวอย่างการคำนวณ

ข้อมูลเข้า (วันทำงาน)	ค่า	การคำนวณ	ผล
register / actual	50 / 45	leave = 50 − 45	5 คน
helpInNormal / helpOutNormal	2 / 1	normal = (45 + 2 − 1) × 8	368 ชม.
ot1Actual / helpInOT1 / helpOutOT1	20 / 3 / 0	ot1 = (20 + 3 − 0) × 3	69 ชม.
manHours			**437 ชม.**
Output วันนั้น	12,000 pcs	productivity = 12,000 ÷ 437	**27.46 pcs/MH**

ตัวอย่างวันหยุด: register 50, actual 30, ot1Actual 0 → `ot2Psn = 30` (เพราะ 30 < 50) → `ot2Hours = 30 × 11 = 330` **ชม.**
กรณีขอบ: ถ้าวันหยุดมีคนมาครบทุกคน (actual = register) และไม่มีใครสแกนออกหลัง 19:30 ระบบจะได้ `ot2Psn = 0` และ Man-hour = 0

5. การรวมกลุ่ม Macro / Direct

5.1 กลุ่มที่รวมจากไลน์ย่อย (บวกค่าทั้ง 7 ตัว)

กลุ่ม	องค์ประกอบ (prefix)	อ้างอิง
LINE B	line_b_gen + line_b_non	`:181-189`
LINE VAC & HPS	line_vac + line_hps	`:197-205`
OQI_M	oqi_m_a + oqi_m_b + oqi_m_d + dqa	`:212-220`
QA SMT (= MQA)	oqi_s_auto + oqi_s_gen + qa_smt_indirect	`:225-242`
QA FPC	oqi_f_auto + oqi_f	`:243-251`
LINE MAS & REW	mas + rew	`:263-272`

MD LAM (combined)	lam_md + md_lam	:318-333
AIX-MOT (= SMT-MOT)	mota + motb	:346-355
ASY SMT	asy1_a + asy1_g + asy2 + asy3 + asy4	:497-510
Macro SMT_F	mota, mota_g, motb, motb_g, motc, motd, astp_a, astp_g, xray, mas, rew, s_tste_f, s_tech_f	:448-461
SMT FRONT_DIRECT	Macro SMT_F ลบ s_tste_f, s_tech_f	:464-478
SMT FRONT_INDIRECT	s_tste_f + s_tech_f	:480-494
Macro SMT_B	blk_1, blk_2, asy1_a, asy1_g, asy2, asy3, asy4, aelt, smt_lam, lam_md, md_lam, s_tste_b, s_ind	:512-525
SMT BACK_DIRECT	Macro SMT_B ลบ s_tste_b, s_ind	:528-542
Macro SMT	Macro SMT_F + Macro SMT_B	:561-569
Direct SMT	Macro SMT − (s_tste_f + s_tech_f + s_tste_b + s_ind)	:572-588
Custom macro	ไลน์ใน `custom_macro_lines.sub_lines` (แปลงรหัส CC เป็นชื่อไลน์ผ่าน `cost_centers` ก่อน)	:617-647

5.2 กลุ่มที่คำนวณจากยอดรวมโรงงานโดยตรง

Macro FPC, Direct FPC และ Macro PCN ไม่ได้รวมจาก calcSubline แต่ใช้ตัวนับรวม (`fpc_*` , `total_*`) พร้อมสูตรของตัวเอง:

```
// Macro FPC  (:396-419)
reg = fpc_total_register;  act = fpc_total_actual;  ot1 = fpc_ot1_actual
help = ผลรวม help ของ 15 prefix FPC (14 direct + line_mat)
normal = (act + helpIn − helpOut) × 8          (วันทำงาน)
ot1H   = (ot1 + helpInOT1 − helpOutOT1) × 3     (วันทำงาน)
ot2Psn = act > 0 ? act : ot1                     (วันหยุด: ต่างจาก calcSubline)
ot2H   = (ot2Psn + helpInOT2 − helpOutOT2) × 11
leave  = reg − act                               (ไม่มี max(0), ไม่ปัดเศษ)

// Direct FPC (:422-445)
reg = fpc_direct_register; act = fpc_direct_actual; ot1 = fpc_direct_ot1_actual
help = ผลรวมของ 14 prefix direct (ไม่รวม line_mat)
ot2Psn = ot1 > 0 ? ot1 : (act < reg ? act : 0)  (กฎเดียวกับ calcSubline)

// Macro PCN (:591-614): สูตรเดียวกับ Macro FPC แต่ใช้ total_* และ help_* ระดับโรงงาน
```

prefix ของ FPC direct: line_hps, line_vac, line_blk, line_lam, line_ost, line_avi_k2, line_d, line_c, line_b_gen, line_b_non, at_front, at_vac, at_lam, line_a_final

6. Output และ Plan

6.1 แหล่ง Output

แหล่ง	การแปลง
`output_daily_snapshot`	รวม `actual_piece_qty` , `actual_sht_qty` , `actual_lot_qty` ต่อ (วัน, line_group) ทุก process และทุก mc_line; ชื่อที่มี "AVI" รวมเป็น `AVI/K2` ; ชื่อ NPM ทุกแบบรวมเป็นชุดเดียวกัน :654-675
`mat_daily_output`	LINE MAT: `piece += pd_output` , `sht += mos_output` :678-693

6.2 Output ของกลุ่ม (getLineOutputForDate)

กลุ่ม	Output ที่นำมารวม	อ้างอิง
Macro PCN	NPM, LINE A, LINE B_GEN, LINE B_NON, LINE C, LINE D, ASY1_A/ASY1_G, ASY2, ASY3, AIX-ASY	:865-867
Macro FPC, Direct FPC	NPM, LINE A, LINE B_GEN, LINE B_NON, LINE C, LINE D	:868-870
Macro SMT, Direct SMT, ASY SMT, Macro SMT_B, SMT BACK_DIRECT	**เฉพาะ ASY**: ASY1_A, ASY1_G, ASY2, ASY3, AIX-ASY	:875-877
Macro SMT_F, SMT FRONT_DIRECT	MOTA_A, MOTA_G, MOTB, AIX-MOT, MOTC, MOTD	:878-880
LINE B / MAS & REW / QA FPC / QA SMT	B_GEN+B_NON / MAS+REW / OQI_F-AUTO+OQI_F-GEN / OQI_S-AUTO+OQI_S-GEN	:884-923
LINE VAC & HPS	ใช้ Output ของ LINE VAC อย่างเดียว	:893-895
Custom macro	ไลน์ใน `output_source_line`	:955-969

แนวคิดคือ Output ของ Macro นับที่ **ไลน์ปลายทาง** (Final) เท่านั้น ส่วน Man-hour นับทุกไลน์ในกลุ่ม เช่น Productivity ของ Macro SMT = Output ของ ASY ÷ Man-hour ของ SMT Front + Back

ในฟังก์ชัน `sumLines()` ชื่อไลน์ที่ normalize แล้วซ้ำกันจะถูกนับครั้งเดียว (`processedCanonical`) กันการนับ ASY1_A กับ ASY1_AUTO ซ้ำ

6.3 Plan

- **SMT** (`smt_daily_actual_plan`): มีเฉพาะ `piece` ถ้ามีหลายแถวที่ normalize แล้วเป็นไลน์เดียวกัน แถวที่มีค่ามากกว่า 0 จะแทนที่ :733-744
- **FPC** (`fpc_daily_actual_plan`): มี piece, sht, lot และ **แทนที่ SMT เสมอ**ถ้าไลน์ชนกัน :746-757
- กลุ่มที่ Plan เป็นผลรวม: MAS & REW = MAS + REW, LINE B = B_GEN + B_NON, QA FPC, QA SMT :779-801
- `piece_plan` ของช่วงเวลา = ผลรวม Plan รายวันทุกวันในช่วง (รวมวันหยุด)

7. การรวมผลรายช่วงเวลา (Period Summary)

7.1 ประเภทช่วงเวลา

ประเภท	นิยาม
DAILY	เฉพาะวันที่ปิดแล้ว (น้อยกว่าวันนี้) เพราะ Output ของวัน D ปิดยอดวันที่ D+1 เวลา 09:00 :1028-1046
WEEKLY	ISO week ที่ถูกตัดตามขอบเดือน คีย์ `YYYY-MM-Www` จาก `getProductionWeekKey()` : ถ้าวันที่ ≥ 20 และเดือนนั้นมีวันในสัปดาห์นี้ ≤ 2 วัน หรือวันที่ ≤ 7 และมี ≤ 1 วัน ให้ยกวันเหล่านั้นไปรวมกับเดือนของวันพฤหัสบดี :19-47, :1049-1081
MONTHLY	วันที่ 1 ถึงสิ้นเดือนปฏิทิน
YEARLY	1 ม.ค. ถึง 31 ธ.ค.

ตัวอย่าง: สัปดาห์ ISO W31/2026 (27 ก.ค. ถึง 2 ส.ค.) แยกเป็น `2026-07-W31` (27–31 ก.ค.) และ `2026-08-W31` (1–2 ส.ค.)

7.2 สูตรต่อ (ช่วงเวลา, ไลน์)

วนทุกวันในช่วง โดยนับเฉพาะวันที่ `register > 0` หรือ `manHours > 0` เป็น "วันที่มีข้อมูล" :1153-1184

```
D  = จำนวนวันที่มีข้อมูล;  Dw = วันทำงานที่มีข้อมูล;  Dh = วันหยุดที่มีข้อมูล

op_leader_register     = round( Σregister / D )
total_man_hour         = round2( ΣmanHours )
leave_psn              = DAILY ? Σleave : round( Σleave / Dw )
leave_working_day_rate = Σleave / Σregister(วันทำงาน) × 100
ot_psn                 = round( ΣotPsn / D )
ot_working_day_rate    = ( ΣotWorking / Dw ) / op_leader_register × 100
ot_holiday_day_rate    = ( ΣotHoliday / Dh ) / op_leader_register × 100

actual_productivity     = Σpiece_output / ΣmanHours        (pcs/MH)
sht_actual_productivity = Σsht_output   / ΣmanHours        (sht/MH)

sum_productivity:
  DAILY            → Σpiece / ΣMH ตั้งแต่วันที่ 1 ของเดือนถึงวันนั้น (MTD)
  WEEKLY, MONTHLY  → Σpiece / ΣMH ตั้งแต่ 1 เม.ย. ของปีงบถึงวันสิ้นสุดช่วง (FYTD)
  YEARLY           → = actual_productivity
  (นับเฉพาะวันที่ MH > 0 และมี Output > 0; ถ้าไม่มีเลยใช้ actual_productivity)
```

อ้างอิง :1186-1256 ปีงบ: ถ้าเดือนของวันเริ่มช่วง ≥ เม.ย. ใช้ปีนั้น ไม่เช่นนั้นใช้ปีก่อนหน้า ค่าทั้งหมดปัดเป็นทศนิยม 2 ตำแหน่งตอนบันทึก :1296-1323

ข้อควรรู้เรื่องตัวหาร: `op_leader_register` เฉลี่ยจากทุกวันที่มีข้อมูล (รวมวันหยุด) แต่ `ot_working_day_rate` ใช้ตัวเศษเฉลี่ยเฉพาะวันทำงาน ถ้าวันหยุดมี Register ต่ำ อัตรา OT วันทำงานจะสูงกว่าที่ควรเล็กน้อย

8. Parser ของ Excel และ Snapshot

8.1 การนำเข้า Attendance รายเดือน (monthlyAttendanceService)

เขียนทับ `productivity_attendance_daily_snapshot` ของเดือนนั้นทั้งเดือนใน transaction เดียว แต่ใช้กฎที่ต่างจากการคำนวณสด:

หัวข้อ	คำนวณสด (attendanceCompute)	นำเข้ารายเดือน
Actual วันทำงาน	`work_status ∈ {Normal, Abnormal}`	มีการสแกน หรือหมายเหตุมีคำว่า NORMAL; ถ้าหมายเหตุมี ลา / LEAVE / ABSENT / ขาด ถือว่าไม่มา :647-668
Help ปกติ	`hour / 8` (คนเทียบเท่า) ทุก type	`hour` (ชั่วโมงดิบ) เฉพาะ type = 'NORMAL' :811-823
Help OT1 / OT2	จำนวนคนไม่ซ้ำ	ผลรวมชั่วโมง เฉพาะ type 'OT 1' หรือ 'OT 2' :824-842
help_in_psn	คนไม่ซ้ำ	จำนวนแถว
total_man_hour	8 × Actual + 2.5 × OT1 ต่อ prefix	8 × Actual + 2.5 × OT1 + ชั่วโมง Help สุทธิ :846-858

8.2 SMT Plan (excelFolderSync)

- อ่านไฟล์ `*smt daily output monitoring*.xlsx` ชีทที่มีคำว่า daily output, assy output หรือ mot-output
- แถวเป้าหมายคือแถวที่มีคำว่า "daily actual plan", "daily plan" หรือ "daily master" และไม่มีคำว่า acc, kpr, balance แล้วย้อนขึ้นไปหาแถววันที่ที่ใกล้ที่สุด :328-371
- ชื่อไลน์มาจากเซลล์แรกของแถว หรือชื่อไลน์ล่าสุดที่เห็นใน 5 คอลัมน์แรก แล้วแปลงด้วย `getImportTargetLines()` (เช่น AIXZ ในชีท ASSY → AIX-ASY และ AIX-BLK)
- ค่า Plan ถูกปัดเป็นจำนวนเต็ม และรับเฉพาะค่า ≥ 0; เดือนได้จาก prefix ชื่อไฟล์ (JAN-...); ถ้าไม่พบใช้ '07'

8.3 FPC Plan (fpcExcelSync + fpcPlanMapping)

- 16 รายการตาม `FPC_TARGET_SPECS` : หาชีทตาม matcher → หาแถว Section จากคำค้น (ข้ามแถวที่มีคำยกเว้น) → ค้นแถว Plan ภายใน 75 แถวถัดไป
- แถวที่มีคำว่า diff, variance, actual (ยกเว้น actual plan), acc, balance, wip, forecast, smt, /a, /b, amt, baht ถูกข้าม `fpcExcelSync.js:469-489`
- ให้คะแนนแถว pcs: "daily act plan_pcs" / "plan output (pcs)" = 100, "act plan" / "plan output" = 80, "plan" = 60 แล้วเลือกแถวที่ได้คะแนนสูงสุด `:500-520`
- LINE BLK = Plan LOT ของ BLK + PIC; LINE OST = OST_B + OST_C + OST_D (ปัดทศนิยม 1 ตำแหน่ง) ส่วนรายการอื่น lot ปัดทศนิยม 2 ตำแหน่ง และ pcs, sht ปัดเป็นจำนวนเต็ม `:614-653`
- คู่ที่ใช้ Spec เดียวกัน จึงได้ Plan เท่ากัน: Macro FPC = Direct FPC, LINE A = LINE A_FINAL, LINE VAC & HPS = LINE VAC

8.4 LINE MAT Output (matExcelSync)

- ไฟล์ล่าสุด (ตาม mtime) ที่ชื่อขึ้นต้นด้วย "File Scan mat" ชีท `Output Report` : แถว `SPDLINEMAT` → PD, แถว `SMLINEMAT` → MOS, วันที่เริ่มคอลัมน์ F, เดือนหลักอ่านจากเซลล์ E3
- ค่าถูกปัดเป็นจำนวนเต็ม ≥ 0 และ **ไม่เขียนทับด้วย 0** (ถ้าค่าใหม่เป็น 0 จะคงค่าเดิมไว้) `:265-285`

8.5 Snapshot พนักงานและกฎเวลา (snapshotManager)

- **Cutoff 09:00:** ก่อน 09:00 วันล่าสุดที่ปิดยอดคือ D−2 หลัง 09:00 คือ D−1 `snapshotManager.js:75-88`
- **ข้อมูลพนักงานของวันใด:** ใช้ snapshot ล่าสุดที่ ≤ วันนั้นถ้ามี ≥ 1,000 คน ไม่เช่นนั้นใช้ snapshot แรกสุด แล้วจึง fallback ไป MariaDB `:431-454`
- **Retention:** ตั้งแต่วันที่ 15 เก็บเดือนนี้และเดือนก่อน ก่อนวันที่ 15 เก็บย้อน 2 เดือน (ให้แก้ Cost Center ย้อนหลังได้ถึงวันที่ 15 ของเดือนถัดไป) `:383-426`
- **Output snapshot ถือว่าไม่ครบ** (คืนค่าว่างเพื่อให้ดึงจาก Oracle ใหม่) ถ้าไม่มีวันปิดยอดล่าสุด หรือ query แบบ ALL แล้วยอด SMT, FPC หรือ QA เป็น 0 `:506-542`

9. ข้อสังเกตและความเสี่ยง

เรียงตามผลกระทบ ข้อที่ระบุว่า "ควรยืนยัน" ขึ้นกับข้อมูลหรือการตั้งค่าจริงที่ตรวจจากโค้ดอย่างเดียวไม่ได้

สูง H1. outputSnapshotService.js ไม่มีฟังก์ชันซิงก์ Output

ไฟล์นี้มี MD5 เดียวกับ `monthlyAttendanceService.js` และ export แค่ `processMonthlyAttendanceImport` กับ `getMonthlyImportLogs` แต่มีโค้ดเรียก `syncDailyOutputSnapshotFromPostgres` และ `cleanupOldOutputSnapshots` จากไฟล์นี้ใน `snapshotScheduler.js:4,31,62`, `snapshotManager.js:654,713` และ `productivitySettingRoutes.js:8,581,668,690,947,1001,1179`

ผลกระทบ: การเรียกจะ throw `TypeError: … is not a function` ทันทีก่อนถึง `.catch()` เช่นที่ `productivitySettingRoutes.js:581` ซึ่งอยู่หลังการ INSERT line group สำเร็จแล้ว ผู้ใช้จึงได้ 500 ทั้งที่ข้อมูลถูกบันทึก ส่วน `POST /sync-output-snapshot` ใช้งานไม่ได้ และ `output_daily_snapshot` ไม่ถูกเติมอัตโนมัติ ทำให้ Productivity ของวันที่ไม่มี snapshot เป็น 0 น่าจะเกิดจากการคัดลอกไฟล์ผิดตอน "first commit" (eaaa7c5)

สูง H2. หน่วยของ Help ในการนำเข้ารายเดือนไม่ตรงกับสูตร Man-hour

การคำนวณสดเก็บ `help_in_normal` เป็นคนเทียบเท่า (`hour/8`) และเก็บ `help_*_ot1/ot2` เป็นจำนวนคน `attendanceComputeService.js:936-1021` แต่การนำเข้ารายเดือนเก็บเป็นชั่วโมงดิบ `monthlyAttendanceService.js:811-842` ขณะที่ Period Summary นำค่าเหล่านี้ไปคูณ 8, 3 และ 11

ผลกระทบ: เดือนที่นำเข้าจาก Excel พนักงานที่ถูกยืมตัว 8 ชม. จะถูกนับเป็น 64 MH (ควรเป็น 8) และ OT2 3 ชม. จะเป็น 33 MH ไลน์ที่มีการยืมตัวมากจะมี Man-hour ผิดและ Productivity เพี้ยน นอกจากนี้การจับคู่ type แบบตรงตัว ('NORMAL', 'OT 1', 'OT 2') ต่างจากการคำนวณสดที่ค้นหาคำ ('OT1', 'WORKING', ...)

สูง H3. Direct FPC: Register นับผู้จัดการ แต่ Actual ไม่นับ

`fpc_direct_register` เพิ่มเมื่อ `isFpcDiv` `attendanceComputeService.js:778` แต่ `fpc_direct_actual` ต้องผ่าน `isFpcDiv && !isMgrDept` `:817` (ไฟล์นำเข้ารายเดือนเป็นแบบเดียวกันที่ `:693 / :731`)

ผลกระทบ: Leave ของ Direct FPC สูงเกินจริงทุกวันเท่ากับจำนวนพนักงาน Direct ในแผนก `(MGR)` และอัตราลาเพี้ยนตาม ส่วน Macro FPC ไม่มีปัญหานี้เพราะตัดออกทั้งคู่

สูง H4. pd_output และ mos_output ไม่ถูกอัปเดตเมื่อคำนวณซ้ำ

คำสั่ง `ON CONFLICT … DO UPDATE SET` ของ `productivity_period_summary` ไม่มี `pd_output` และ `mos_output` `periodSummaryService.js:1274-1295`

ผลกระทบ: ค่า PD และ MOS ของ LINE MAT จะค้างอยู่ที่ค่าตอน INSERT ครั้งแรก เช่น ถ้า rollup รายเดือนครั้งแรกเกิดตอนต้นเดือน ค่ารายเดือนจะไม่ขยับอีก (piece/sht ไม่มีปัญหาเพราะอยู่ในรายการ UPDATE) และ `sht_prod_target` ถูกเขียนเป็น 0 ทุกครั้ง `:1313`

กลาง M1. OT1 คิด 2.5 ชม. ในจุดหนึ่ง แต่ 3 ชม. ในอีกจุด

`*_total_man_hour` ใน snapshot ใช้ +8 และ +2.5 `attendanceComputeService.js:793,853`, `monthlyAttendanceService.js:846-857` ส่วน Period Summary และ Frontend ใช้ 3 `periodSummaryService.js:141`, `useMatrixAggregation.ts:58` ค่าราย Cost Center (`cc_*_total_man_hour`) จึงใช้ 2.5 และไม่รวม Help ทำให้ Man-hour รายแผนกไม่ตรงกับรายไลน์ ควรกำหนดค่าคงที่เป็นจุดเดียว

กลาง M2. OT2 วันหยุดของ Macro FPC / Macro PCN ใช้กฎต่างจากจุดอื่น

Backend ใช้ `ot2Psn = act > 0 ? act : ot1` periodSummaryService.js:408,603 แต่ Frontend ใช้ `fpcOt1` useMatrixAggregation.ts:339 และไลน์ย่อยกับ Direct FPC ใช้ `ot1 > 0 ? ot1 : (act < reg ? act : 0)` คอมเมนต์ที่บอกว่าตรงกับ Frontend "100%" จึงไม่จริงสำหรับวันหยุด นอกจากนี้ Macro FPC, Direct FPC และ Macro PCN ไม่มี `max(0, …)` และไม่ปัดเศษ ต่างจากไลน์ย่อยและ Frontend

กลาง M3. Output ของ Macro SMT / Macro SMT_B มีโค้ดที่ทำงานไม่ถึงและเรียกฟังก์ชันที่ไม่มีอยู่

เงื่อนไขที่ :875 ดัก Macro SMT, Direct SMT, Macro SMT_B และ SMT BACK_DIRECT ไว้ก่อน จึงคืนค่าเฉพาะ ASY กิ่ง :902 (SMT_B รวม BLK, AELT, SMT_LAM, MD LAM) และ :905-914 (Macro SMT = F + B, Macro PCN = SMT + FPC) ทำงานไม่ถึง และ :905-914 เรียก `resolveDayOutput` ที่ไม่ได้ประกาศไว้ ถ้ามีคนสลับลำดับเงื่อนไข ระบบจะเกิด ReferenceError **ควรยืนยัน** ว่าตั้งใจให้ Output ของ Macro SMT_B เป็น ASY อย่างเดียวหรือไม่

กลาง M4. นิยามวันหยุดไม่ตรงกัน และวันที่จาก tbl_date อาจเลื่อน 1 วัน

`work_day_type` ใช้ ส./อา. กับเกณฑ์ OT1 ≥ 20 คน attendanceComputeService.js:880-893 แต่สูตร Man-hour ใช้ `tbl_date` และถ้าไม่พบในปฏิทินจะถือว่าเฉพาะวันอาทิตย์เป็นวันหยุด periodSummaryService.js:115,125 นอกจากนี้ `parseDate()` แปลง Date ด้วย `toISOString()` :11 ถ้า driver MariaDB คืนค่า `w_date` เป็น Date เวลาเที่ยงคืนตามเวลาไทย (UTC+7) วันจะเลื่อนไปเป็นวันก่อนหน้า และวันหยุดจะผิดวัน **ควรยืนยัน**การตั้งค่า `dateStrings` ของ pool

กลาง M5. FYTD ขาดข้อมูลช่วง เม.ย.–ธ.ค. ของปีก่อน และปีถูก hardcode เป็น 2026

ระบบโหลดข้อมูลตั้งแต่ `${targetYear}-01-01` เท่านั้น :58-59 แต่ปีงบเริ่ม 1 เม.ย. :1237-1238 ดังนั้น `sum_productivity` ของช่วง ม.ค.–มี.ค. จะคิดจาก ม.ค. เท่านั้น ไม่ใช่ตั้งแต่ เม.ย. ปีก่อน และเมื่อเรียกโดยไม่ส่ง startDate (เช่นจาก Scheduler) `targetYear` จะเป็น 2026 เสมอ ในปี 2027 ช่วงที่คำนวณจะยาวขึ้นเรื่อย ๆ

กลาง M6. ข้อมูลสัปดาห์ที่ 31 ถูกสร้างขึ้นและบันทึกถาวร

ถ้าวันที่ 27–30 ก.ค. 2026 ไม่มีข้อมูล ระบบจะคัดลอก metrics ของ 31 ก.ค. (หรือ 24 ก.ค.) มาใส่ attendanceComputeService.js:1025-1033, periodSummaryService.js:102-110 และเพราะ `getAttendanceSummaryData()` บันทึกผลของวันที่ผ่านไปแล้วกลับลง snapshot ค่าที่คัดลอกจึงกลายเป็นข้อมูลถาวร

กลาง M7. Alias ของไลน์ทับกันเอง

`'LINE MD LAM'` ถูกตั้งเป็นค่ารวม lam_md + md_lam :328 แล้วถูกทับเป็น md_lam อย่างเดียว :384; `'ASY5'/'ASY6'/'ASY7'` ชี้ไปที่ asy1_a/asy2/asy3 :359-361 ขณะที่ `'LINE ASY5..7'` ชี้ไป asy5..7 :303-305 และ ASY5–7 อยู่ใน `standardSmtLines` ผลสรุปของ ASY5 จึงเป็นข้อมูลซ้ำกับ ASY1_A ส่วน `'ASY SMT'` ถูกกำหนดสองครั้ง :308, :510

ต่ำ L1. Help ประเภท OT ถูกนับเป็น Help ปกติด้วย (ควรยืนยัน)

ทุกแถวที่ `hour > 0` ถูกบวก `hour/8` เข้า `help_*_normal` โดยไม่ดู type attendanceComputeService.js:938-963 ถ้า `tbl_help.hour` ของแถว OT1 เป็นชั่วโมง OT ชั่วโมงเหล่านั้นจะถูกนับซ้ำทั้งในส่วนปกติ (×8) และ OT1 (×3)

ต่ำ L2. การหาเดือนจากชื่อไฟล์ FPC อาจผิด (ควรยืนยัน)

`detectYearMonthFromFileName()` ใช้ `includes()` เรียงจาก ม.ค. ถึง ธ.ค. fpcExcelSync.js:35-40 ชื่อไฟล์ที่มีคำอย่าง "Summary" จะเจอ "mar" ก่อนและถูกตีเป็นเดือน 03 ควรจับคู่แบบขอบคำ (word boundary) หรือให้เลขนำหน้าไฟล์มาก่อน

ต่ำ **L3. Scheduler ไม่ถูกเรียกใช้ และมีสองชุด**

`initSnapshotScheduler()` มีทั้งใน `snapshotScheduler.js` และ `snapshotManager.js` แต่ไม่มีที่ใดเรียก งาน 09:00 (ซิงก์ Excel, snapshot พนักงาน, rollup) จึงไม่รันอัตโนมัติ ถ้าเปิดใช้ตอนนี้ก็จะติด H1

ต่ำ **L4. ประสิทธิภาพของ rollup**

Period Summary คำนวณใหม่ทั้งปีทุกครั้ง โดย INSERT ทีละแถวต่อ (ช่วงเวลา × ไลน์) และ `sum_productivity` รายวันวนย้อนตั้งแต่ต้นเดือนทุกวัน (O(วัน² × ไลน์)) นอกจากนี้ `saveFpcPlanRecords()` สั่ง rollup ทั้งปีทุกครั้งที่บันทึก Plan `fpcExcelSync.js:701`

ต่ำ **L5. โค้ดค้างและเงื่อนไขที่ไม่มีผล**

- `attLoanStatusMap` ถูกสร้างแต่ไม่ถูกใช้ `attendanceComputeService.js:283, 382-389`
- เทียบ `uLine` ซึ่งเป็นตัวพิมพ์ใหญ่แล้วกับ `'SMT Front_Direct'` จะไม่เป็นจริงเลย `periodSummaryService.js:878`
- การหา OT1 ไม่ครอบคลุมกะกลางวันที่ออกหลังเที่ยงคืน และกะกลางคืนที่ออกหลัง 12:59
- MAT ไม่สามารถแก้ค่าให้กลับเป็น 0 ได้ `matExcelSync.js:269-276`; SMT Plan ใช้เดือน '07' เมื่ออ่านชื่อเดือนจากไฟล์ไม่ได้ `excelFolderSync.js:280`

ลำดับการแก้ไขที่แนะนำ

1. กู้ `outputSnapshotService.js` ตัวจริงจาก repo ต้นทาง (`gitlab.com/fujikura-k1/smart-backend-k1`) หรือเขียน `syncDailyOutputSnapshotFromPostgres` / `cleanupOldOutputSnapshots` ใหม่ (H1)
2. ทำหน่วยของ Help ในการนำเข้ารายเดือนให้ตรงกับการคำนวณสด แล้วนำเข้าเดือนที่ได้รับผลกระทบใหม่ (H2)
3. เพิ่ม `!isMgrDept` ที่ `fpc_direct_register` ทั้งสองไฟล์ แล้วสั่ง sync attendance ย้อนหลัง (H3)
4. เพิ่ม `pd_output` , `mos_output` ใน UPDATE SET (H4)
5. รวมค่าคงที่ 8 / 3 / 11 และกฎ OT2 ไว้ในโมดูลเดียวที่ใช้ร่วมกัน (M1, M2) เพราะ CLAUDE.md ระบุอยู่แล้วว่าสูตรถูกคัดลอกไว้หลายที่

10. ภาคผนวก: ค่าคงที่และกฎการปัดเศษ

ค่าคงที่	ค่า	ใช้ที่
ชั่วโมงงานปกติต่อคน	8	Man-hour ปกติ, แปลง Help hour → คน (hour/8)
ชั่วโมง OT1 ต่อคน (Period Summary)	3	periodSummaryService, Frontend
ชั่วโมง OT1 ต่อคน (Snapshot)	2.5	`*_total_man_hour` , `cc_*`
ชั่วโมง OT2 วันหยุดต่อคน	11	periodSummaryService, Frontend
เกณฑ์ทำงานสั้น (ไม่นับ Actual)	< 4 ชม.	attendanceCompute, monthlyAttendance
เวลาเริ่มนับ OT กะกลางวัน / กลางคืน	19:30 / 07:30	ตรวจ OT1
ช่วงเวลาสแกนเข้าที่ถือเป็นกะกลางคืน	18–23 น.	ตรวจ OT1
เกณฑ์ "ทำงานวันหยุด" (work_day_type = H)	OT1 ≥ 20	attendanceCompute (ไม่ได้ใช้ใน Man-hour)
Snapshot พนักงานที่ถือว่าครบ	≥ 1,000 คน	getEmployeeDataForDate, syncSmartManpower
เวลาปิดยอด Output	09:00 D+1	DAILY period, cutoffYesterdayStr
วันเริ่มปีงบประมาณ	1 เม.ย.	sum_productivity (WEEKLY, MONTHLY)
ประวัติการนำเข้า Excel	7 วัน	`excel_import_history`

ค่า	การปัดเศษ
manHours รายไลน์ย่อยต่อวัน	`Math.round` (จำนวนเต็ม) ส่วน Macro FPC / Direct FPC / PCN / Custom ไม่ปัด
op_leader_register, leave_psn (ไม่ใช่ DAILY), ot_psn	`Math.round`
total_man_hour, productivity, rates	ทศนิยม 2 ตำแหน่ง (`round(x × 100) / 100`)
SMT Plan, FPC pcs/sht, MAT PD/MOS	จำนวนเต็ม และไม่ติดลบ
FPC lot (BLK, OST / อื่น ๆ)	ทศนิยม 1 ตำแหน่ง / 2 ตำแหน่ง

รายงานนี้สร้างจากการอ่านซอร์สโค้ด ณ commit e7e64f5 บน branch Yo ไม่ได้รันกับฐานข้อมูลจริง เลขบรรทัดอ้างอิงไฟล์ใน `smart-backend-88-62/services/productivity/` ยกเว้นที่ระบุชื่อไฟล์อื่น